

Thinking, Faithful and Stable: Mitigating Hallucinations in LLMs & Future Directions

A Process-Level Reinforcement Learning Approach

Presented by Archit Garg

The Situation

- Large Language Models (LLMs) frequently hallucinate during complex, multi-step reasoning tasks.
- Existing mitigation strategies, such as standard RLHF, primarily focus on outcome-level correctness (whether the final answer is right or wrong).
- This outcome-based approach ignores the actual process of hallucination, which usually occurs mid-generation when the model deviates from grounded logic or expresses unwarranted certainty in incorrect claims.

The Task

The Objective:

To develop a self-correcting, process-level reinforcement learning framework that mitigates hallucinations in real-time during multi-step reasoning.

The Strategy:

Instead of optimizing for after-the-fact correctness, the task is to train the model to become more introspective by using fine-grained, continuous indicators of uncertainty.

Key Signals Identified:

- Token-level entropy spikes to detect latent uncertainty
- Self-assessed confidence alignment to detect explicit miscalibration



The Action – Architecture & Formatting

Model & Training:

The researchers fine-tuned a Qwen3-0.6B model using Low-Rank Adaptation (LoRA) and a Group Relative Policy Optimization (GRPO) training loop on the MATH-500 dataset.

Structured Generation:

The system prompt explicitly forced the model to generate its response in three strict segments to promote explainability and introspection:

`<think>...</think>` for concise step-by-step reasoning

`<answer>...</answer>` for the final prediction

`<confidence>...</confidence>` for a scalar confidence score between 0.0 and 1.0

The Action – The Composite Reward Function

The GRPO pipeline utilized a highly specific, three-part reward function to shape the model's behavior:

Self-Confidence Calibration:

Penalizes overconfidence on wrong answers and underconfidence on right answers.

$$R_{\text{confidence}} = (2 \cdot \text{correct} - 1) \cdot (2 \cdot \text{confidence} - 1)$$

Entropy Spike Detection:

Computes token-level Shannon entropy and applies a z-score filter to penalize sudden spikes in uncertainty during the <think> phase.

$$R_{\text{entropy}} = -\lambda \cdot \max(0, z_{\text{max}} - \tau)$$

Format Enforcement:

A hard +1.0 or -1.0 reward to ensure the XML-style tags are strictly followed.

The Result – Performance & Calibration

1

Accuracy

Fine-tuning with this confidence-aligned reward improved overall reasoning accuracy on the MATH-500 subset from 34.0% to 37.0%.

2

Calibration

The model became drastically more self-aware.

- The absolute calibration error dropped from 0.38 to 0.29
- Expected Calibration Error (ECE) fell from 0.42 to 0.19
- The Brier Score improved from 0.22 to 0.11

3

Formatting

Format validity jumped from 52.0% to 96.0%.

The Result – Reasoning Stability & Ablation

Smoother Reasoning:

Token-level entropy metrics proved the model's internal reasoning became more stable.

- Average entropy fell from 0.431 to 0.405
- The standard deviation of entropy dropped from 0.102 to 0.085

Ablation Study:

The researchers proved the signals are complementary. Using only the confidence reward yielded a 36.2% accuracy, but adding the entropy reward pushed it to the full 37.0%.

Efficiency:

These major gains in stability and accuracy were achieved without increasing inference latency, which remained at roughly 3.42 seconds per example.

Our Plan – The Hybrid Pipeline

The Goal:

To replace/augment surface-level statistical rewards (entropy) with deep, structural, internal-state rewards using Graph Neural Networks (GNNs) on attention traces.



Step 1: Generation & Trace Extraction

As the LLM generates reasoning steps, we dynamically extract output probabilities (for entropy), explicit confidence statements, and the internal attention matrices.



Step 2: Dynamic Graph Construction

The attention matrices are sparsified and structured into a directed graph where nodes are tokens and edges are attentional flows.



Step 3: GNN Forward Pass

A pre-trained CHARM network processes the graph using the layer-wise update function to output a continuous hallucination probability score.

GNN layer update (for reference):

$$h_i^{(t+1)} = \text{up}_t \left(h_i^{(t)}, \frac{1}{\text{deg}_{\text{in}}(i)} \sum_{j:(i,j) \in E} \text{msg}_t \left(h_j^{(t)}, x_{E,(i,j)}^\tau, p_{i,j} \right) \right)$$

where $\text{deg}_{\text{in}}(i)$ is the in-degree of node i , E is the edge set, and τ is the sparsification threshold.

Our Plan – The Unified Reward Function

By combining both frameworks, our final reinforcement learning reward will synthesize lexical smoothing, structural logic, and explicit self-awareness.

The Proposed Master Equation:

$$R_{total} = R_{confidence} + R_{entropy} - \gamma \cdot R_{CHARM}$$

Why This is Powerful (Triangulated Feedback):



R_confidence ensures the model's self-assessment is perfectly calibrated to reality.



R_entropy acts as a local smoothing regularizer, preventing wild micro-level guessing.



R_CHARM (derived from the GNN output $\hat{y}$) acts as an internal logic critic, penalizing poor attention distribution and structural representational collapse.

Our Plan – Implementation Challenges

VRAM Constraints:

Extracting attention matrices concurrently during a GRPO rollout requires significant memory. Strict adherence to the sparsification threshold ($\tau = 0.05$) is necessary to make the computational graph viable.

Two-Stage Pipeline:

The CHARM GNN must be pre-trained on a dataset of known computational traces before it can be effectively "frozen" and utilized as the Reward Model in the RL pipeline.

Avoiding Reward Hacking:

Ensuring the LLM doesn't just learn to generate text that produces "healthy-looking" attention graphs to fool the GNN, which is why keeping the R_confidence ground-truth anchor is critical.

References

Thinking, Faithful and Stable: Mitigating Hallucinations in LLMs

[Chelsea Zou](#), [Yiheng Yao](#), [Basant Khalil](#)

arXiv Preprint, 2025

<https://doi.org/10.48550/arXiv.2511.15921>

CHARM: Graph Neural Networks for Detecting Hallucinations in Large Language Models via Attention Trace Analysis

Dr. David Kim, Dr. Emily White, Dr. Frank Green

arXiv Preprint, 2025

<https://doi.org/10.48550/arXiv.2509.24770>

Thank You!

Questions? Let's debug the hallucinations together!



Email

architg575@gmail.com



LinkedIn

/in/architg575



X

@Archit_tech